

1. Java Package:

A java package is a group of similar types of classes, interfaces and sub-packages.

Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

2. If you are using one class in another class of different package then you need to import that class of different package.

3. Properties which are not related to unique objects but are common to all objects are known as static.

For example "population" in "Human" class.

4. Although static variables can be called using object name but by convention we use class name.

```
/*
```

```
*/
```

```
*****
```

5. Inside a "static" method you cannot call "non-static" method directly.

Any method which is non-static must be dependent upon object/instances to call it and static methods are independent of objects.

Hence non-static methods cannot be called inside static methods directly.

```
static void fun() {  
greeting(); // Cannot be called because of the above reason.  
}
```

```
void greeting() {  
System.out.println("Hello");  
}
```

```
*****
```

6. Static methods can be called inside non-static methods directly.

```
static void fun() {  
System.out.println("Fun");  
}
```

```
void greeting() {  
fun(); // Can be called.  
}
```

```
*****
```

7. You can call a "non-static" method inside a "static" method by creating an instance of class to which non-static method belongs.

```
static void fun() {  
Main obj = new Main();  
obj.greeting(); // Can be called because an instance of class is created.  
}
```

```
void greeting() {
System.out.println("Hello");
}
```

8. You can call a non-static method1 inside a non-static method2 directly without creating the instance of method1

because method2 will be somewhere called by creating its class instance. Hence, indirectly method1 is called via instance.

```
void fun() {
greeting(); // Can be called because somewhere instance of fun must be created and
// hence, greeting is called indirectly via instance.
}
```

```
void greeting() {
System.out.println("Hello");
}
```

9. You cannot use "this" inside a static method because this is replaced by objects and hence dependent on objects

but static methods are independent of objects.

10. Static blocks are used to initialize static variables in a class.

As soon as first object of the class is created static block will run and all variables inside static block will be initialized.

```
static {

}
```

11. Why outer classes cannot be static?

We can't declare outer (top level) class as static because the static keyword is meant for providing memory

and executing logic without creating Objects, a class does not have a value logic directly(Class is just a logical instruct),

so the static keyword is not allowed for outer class.

Because it doesn't make any sense. Just think about it. Something XYZ Static means that XYZ something is part of a class, not of an instance.

An outer class can't be part of an instance or another class and not to itself, so what sense does it make to say it's static or not?

A nested class can be static because it is a perfect statement " InnerClass is associated with the OuterClass class and not to instance of OuterClass class"

For more info refer InnerClasses in StaticExample in OOP2.

12. Explain System.out.println();

System is a class.

out is a reference variable or instance of Printstream class which is declared inside System class and is static.

println() is a method of Printstreamclass.

Printstream -> Class

println() -> method of Printstream

out -> reference variable / instance of Printstream as well as static variable of System class.

System ek class hai jiske andar Printstream ka object ya instance banaya hai.

Iss instance ka naam "out" hai aur ye instance static hai.

Kyunki "out" static hai isliye ise class ke naam se yaani 'System' ke through bulaya gaya

hai(System.out)

println Printstream ka method hai aur ise Printstream ke object ya instance se bula skte hai aur isiliye ise "out" se

bulaya hai(out.println).

13. How can you allow to create only one instance of class?

<https://www.simplilearn.com/tutorials/java-tutorial/singleton-class#:~:text=A%20singleton%20class%20in%20Java%20ensures%20only%20one%20instance%20of,if%20not%2C%20it%20creates%20one.>

Refer Singleton class in Singleton package in OOP2.